

Smart Road Vision System (SRVS) Complete Backend Development Plan



Architectural Vulnerabilities & Upgrades

1. The Synchronous Latency Trap

- **The Flaw:** If the system pauses the video feed analysis to wait for a fallback API (Extract_Numberplates.ipynb) or an enhancement API (Real-ESRGAN), the entire pipeline will block. You will drop video frames and lose tracking accuracy.
- **The Fix:** We must explicitly instruct the AI to use an **Asynchronous / Event-Driven Architecture**. Frame processing and tracking must happen on the main thread, while API calls, ESRGAN enhancements, and CSV/Database writing must be dispatched to a background worker queue (e.g., using Python's asyncio, Threading, or a message broker like Celery/Redis).

2. The "Best Snapshot" Ambiguity

- **The Flaw:** Telling an AI to capture the "best snapshot with highest visual detail" is highly subjective. The AI might write a naive loop that just saves every frame or the last frame.
- **The Fix:** We must define the mathematical heuristic for "best." Instruct the AI to evaluate frames based on **BBox (Bounding Box) Area** (larger = closer) and **Laplacian Variance** (higher = less motion blur).

3. The Speeding Physics Missing Link

- **The Flaw:** You asked the AI to detect over-speeding. Pixel displacement across frames cannot determine speed without real-world context.
- **The Fix:** Instruct the AI to implement a **Homography Matrix** or "Perspective Transformation" to map 2D pixel coordinates to real-world distances, combined with a frame-rate-to-time conversion.

4. The File-Locking Hazard

- **The Flaw:** Writing to a single CSV file dynamically in a highly concurrent tracking environment will result in file-locking errors and data corruption when multiple threads attempt to write different Tracking_IDs simultaneously.
- **The Fix:** Instruct the AI to handle CSV appends via a thread-safe queue, or better yet, use an in-memory database like SQLite that flushes to CSV periodically.

Core Dependency Matrix

1. Asynchronous I/O & Concurrency

- **aiofiles**: Required for non-blocking, thread-safe asynchronous reading/writing to CSV files.
- **httpx**: A fully async HTTP client. Essential for triggering the API-based fallbacks (Extract_Numberplates, Extract_Helmets, and Real-ESRGAN) without blocking the main video processing loop.

2. The Vision & Deep Learning Engine

- **torch & torchvision**: The foundational deep learning framework required by both YOLOv8-OIV7 and TransReID.
- **ultralytics**: The official library for running the YOLOv8-OIV7 object detection models (for initial vehicle and bike tracking).
- **opencv-python-headless**: The core computer vision library for frame extraction, BBox rendering, and Homography Matrix calculations (speed tracking). *Using the headless version is mandatory for Jules's cloud VM environment to prevent X11 server errors.*
- **Pillow (PIL)**: Required for image transformations and handling the cropped snapshots before passing them to the APIs or saving them.

3. TransReID Specifics

- **timm (PyTorch Image Models)**: TransReID relies heavily on pre-trained vision transformer models housed in this library.
- **yacs**: Used by TransReID for managing complex configuration setups (loading the .yaml files that define the ReID model's weights and architecture).
- **scipy**: Required by TransReID for calculating distance metrics (like Cosine Similarity or Euclidean distance) to match vehicle features across frames.

4. OCR Engine (Local Fallback/Primary)

- **paddlepaddle-gpu** (or paddlepaddle for CPU): The core framework for PaddleOCR.
- **paddleocr**: Required if you are running the OCR locally alongside YOLO before failing over to the API.

5. Data & Matrix Operations

- **numpy**: Essential for handling the OpenCV frame arrays and calculating the Laplacian Variance (motion blur heuristic).
- **pandas**: Useful for structuring the final data logs if you choose to batch-process the CSV writes.

System Prompt for Autonomous Execution

Objective: Develop and autonomously deploy the complete backend pipeline for the Smart Road Vision System (SRVS) Project. The system utilizes a dual-stream architecture, TransReID for spatial-temporal vehicle re-identification, local baseline YOLOv8s-oiv7/PaddleOCR inferencing, and asynchronous API-based fallbacks for edge-case resolutions. The architecture must be highly concurrent, thread-safe, and capable of real-time processing without frame-dropping.

Phase 1: Pre-Execution Environment Management

- **Python 3.12 Standardization:** Provision the virtual environment utilizing Python 3.12 to optimize asyncio loop performance and ensure pre-compiled binary compatibility for deep learning libraries. This version serves as the architectural sweet spot, offering the optimal balance of performance primitives and dependency compatibility for real-time vision processing.
- **Headless Enforcement (CRITICAL):** Override any existing requirement.txt requests for standard opencv-python and install opencv-python-headless. Autonomously strip or disable any GUI-blocking code (cv2.imshow(), cv2.waitKey()) to prevent fatal X11 server crashes in the cloud VM.
- **Notebook Conversion Protocol:** The fallback modules (Extract_Numberplates.ipynb, Extract_Helmets.ipynb, Enhanced_Image_Generation_Real-ESRGAN.ipynb) are currently in Jupyter Notebook format. Autonomously convert these into standard .py modules. Refactor their global execution logic into callable, asynchronous functions so they can be safely imported and executed by the main pipeline without blocking the event loop.
- **OIV7 Model Integration:** Do not use yolov11n.pt. Initialize the detection engine using yolov8s-oiv7.pt. The Ultralytics engine will automatically download the Open Images V7 weights. Update the class filtering dictionaries to map the specific OIV7 class indices for Cars, Trucks, Buses, Motorcycles, and Auto-rickshaws.
- **Dependency Audit:** Ensure aiofiles, httpx, torch, ultralytics==8.3.0, timm, yacs, and paddleocr are installed. Use the standard library sqlite3 for thread-safe data handling.

Phase 2: Dual-Stream Input Ingestion & Separation of Concerns

Initialize two distinct asynchronous video feed processors using placeholders for the video sources. Maintain strict logical separation between them:

- **Stream A (Front-Facing):** Dedicated to incoming large traffic. Filter detections for standard OIV7 classes (Cars, Trucks, Buses). Process tracking, TransReID, and ANPR. Ignore Helmet Detection. Ignore Motorcycles and Auto-rickshaws.
- **Stream B (Rear-Facing):** Dedicated strictly to outgoing traffic, explicitly filtering for Motorcycles and Auto-rickshaws. Process tracking, TransReID, and ANPR. *Execute Helmet Detection for Motorcycles only; skip Helmet Detection for Auto-rickshaws; ignore other vehicles.*

Phase 3: Two-Stage Tracking & Heuristic Snapshot Extraction

- **Two-Stage Architecture Enforcement:** Strictly utilize the two-stage detection architecture demonstrated in the local numberplate.py and helmet.py files (Detect Vehicle Bounding Box → Crop → Extract Features). Disregard the single-stage architecture found in video.py, as full-vehicle bounding boxes are mandatory for TransReID feature extraction.
- **Tracking & ID Generation:** Assign a unique Tracking_ID to every distinct vehicle across frames using TransReID. Implement spatial-temporal track retirement (e.g., a line-crossing event or a timeout) to finalize a vehicle's state and prevent duplicate IDs.
- **Heuristic Snapshot Extraction & "Best Snapshot" Logic:** Do not save every frame. Maintain a rolling buffer for each active Tracking_ID. Extract the single best cropped snapshot for OCR evaluation by calculating:
 - Maximum Bounding Box area (vehicle is closest to the camera).
 - Highest Laplacian Variance (image is the sharpest/least blurred).

Phase 4: The 3-Tier Plate Reading & Violation Engine

Once a vehicle's track is finalized and the best snapshot is extracted, execute the following strict sequence for plate extraction. Rely entirely on this fallback logic to catch edge-cases, misclassifications, and camouflaged plates:

- **Tier 1 (Baseline):** Execute local YOLOv8-OIV7/PaddleOCR plate extraction on the snapshot.
- **Tier 2 (API Fallback):** If Tier 1 fails to detect a plate, or returns an unreadable/low-confidence string, autonomously dispatch the exact same snapshot to the newly converted, asynchronous Extract_Numberplates API module.
- **Tier 3 (Final State):** If both local and API approaches fail, explicitly log the Read_Number_Plate field as "Missing/Obstructed" and flag Violation_Detected as True.
- **Helmet Detection (Stream B Only):** Apply a similar local-to-API fallback logic for helmet detection on motorcycles.
- **Over-Speeding:** Implement a perspective transformation (Homography Matrix) with placeholder reference points to map pixel-displacement to real-world meters. Calculate speed using displacement over time ($v = d/t$ based on FPS).

Phase 5: Thread-Safe Storage & Data Persistence

- **Images:** Save the final evaluated snapshot in separate directories by class using the format: `f"{Tracking_ID}.jpg"`.
- **Data:** Implement thread-safe data logging using the built-in `sqlite3` module (or `aiofiles` for CSVs) to prevent file-locking during concurrent tracking threads.
- **Database Schema A (Motorcycles):** `Tracking_ID`, `Read_Number_Plate`, `Helmet_Detected` (Boolean), `Violation_Detected` (Boolean).
- **Database Schema B (Auto-Rickshaws):** `Tracking_ID`, `Read_Number_Plate`, `Violation_Detected` (Boolean).
- **Database Schema C (Large Vehicles):** `Tracking_ID`, `Read_Number_Plate`, `Violation_Detected` (Boolean).

Phase 6: Post-Processing Enhancement Pipeline

- Once a Stream B (Motorcycle) track is finalized and its snapshot is saved to disk, dispatch an asynchronous task to the converted `Enhanced_Image_Generation_Real-ESRGAN` API module.
- Restore the visual quality of the snapshot for dashboard deployment.
- Save this output in a dedicated directory formatted as: `f"Restored_{Tracking_ID}.jpg"`.